QuickCart technical documentation

A Multi Container Basic E Commerce Checkout System with Jenkins CI CD Pipeline

System Architecture and Integration | October 2026

QuickCart combines a React storefront, three Express APIs and PostgreSQL in Docker. Jenkins is configured separately to test, build and deploy changes from a shared Git repository.

Local verification on 9 October 2026: all 76 API tests passed, frontend lint passed and the Vite production build passed. Live Docker and Jenkins evidence remains pending because this assistant environment cannot access the Docker engine.

The group name, course section and shared repository URL still require the group's confirmation. This report describes the PostgreSQL implementation and the procedures for completing the required live demonstrations.

Member	Assigned role
Cinense, Kyle Mathew T.	DevOps and Project Manager
Cuenco, John Gabrielle D.	Frontend Lead
Dayrit, Shaira Benneth M.	QA Documentation and Presentation
Frayre, Angeline B.	Database and Orchestration Lead
Perez, Jan Andre T.	Backend Microservice Lead

Project description and modules

QuickCart is a demonstration gift shop. A shopper browses products, searches or filters the catalog, adds products to a cart, supplies delivery details and completes checkout using simulated payment methods. My Account finds orders by email, and the contact page stores support tickets.

The application has four custom application modules: frontend, catalog API, order API and payment API. The reverse proxy and database provide supporting infrastructure. This exceeds the guideline requirement of three application modules in separate containers.

Standard shipping is free. Express shipping costs $14.99. A subtotal of at least $100 receives a $10 discount. The server calculates prices and totals from current database records; browser supplied prices are not used.

Payments are simulated. The demo has no password based account login or real payment integration. Order lookup by email is suitable for classroom demonstration and does not provide account authentication.

Module	Responsibility
frontend	React pages, cart state and API requests
catalog-api	Products, search suggestions and categories
order-api	Quotes, checkout, history and support
payment-api	Mock approval, decline and card validation
proxy	Single public gateway and path routing
db	Products, customers, orders, items, logs and tickets

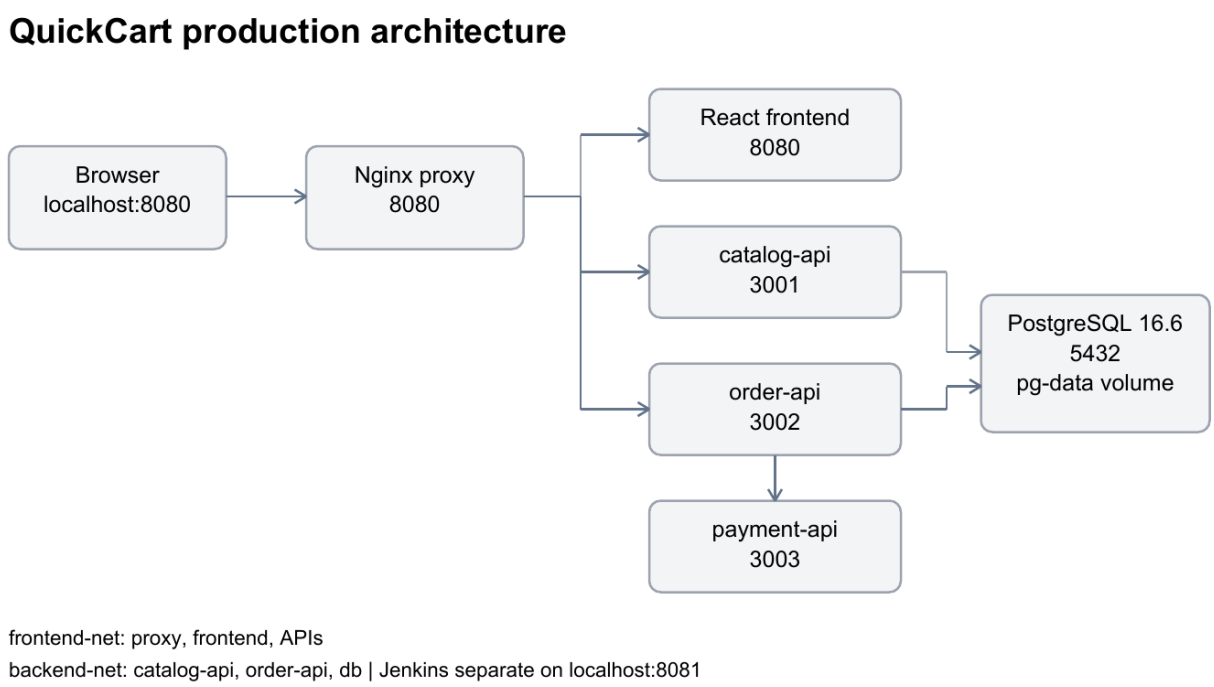
Architecture and request flow

Production traffic enters Nginx at localhost:8080. Nginx forwards the storefront to frontend:8080, catalog requests to catalog-api:3001, and orders/support requests to order-api:3002. Only the payment health endpoint is routed publicly; order-api calls the internal payment charge endpoint.

frontend-net connects the proxy, frontend and APIs. backend-net connects catalog-api, order-api and PostgreSQL. The database is not attached to frontend-net and has no published host port. PostgreSQL listens internally on 5432.

Checkout reads product prices and stock, obtains a simulated payment result, then writes the order and stock changes in a PostgreSQL transaction. Conditional stock updates protect against concurrent purchases. Database failure rolls back the database writes.

In host development, Vite owns localhost:8080 and forwards /api to the Docker proxy at 127.0.0.1:8082. Production uses the base Compose file, where the proxy owns 8080. Jenkins is a separate stack on localhost:8081.



Resource	Purpose
pg-data	Persistent PostgreSQL data at /var/lib/postgresql/data
jenkins_home	Persistent Jenkins jobs, plugins and credentials
Docker socket	Allows the Jenkins container to build and deploy on the host

Technology stack and justification

PostgreSQL is the database management system. SQL is the query language used to define the six tables and read or write records. The current database image is postgres:16.6-alpine. The previous MySQL configuration is not used by the current application.

Catalog and order repositories use the pg driver, parameterized $1 queries, RETURNING, identity columns and PostgreSQL transactions. Money calculations use integer cents in application logic, while persisted monetary columns use DECIMAL(10,2).

Dependency lockfiles and npm ci provide repeatable JavaScript dependency installation. Base images use versioned tags rather than latest, although several tags remain mutable major/minor tags rather than digest pins.

Component	Choice	Reason
Frontend	React and Vite	Components and compiled static assets
APIs	Node.js and Express 5	Shared JavaScript stack and small services
Database	PostgreSQL 16.6	Relational constraints and transactions
Gateway	Unprivileged Nginx 1.27	Path routing and non-root runtime
Orchestration	Docker Compose	Services, networks and named volumes
CI/CD	Jenkins LTS	Declarative pipeline stored in Git
Tests	Jest Supertest ESLint	Logic, route and code quality checks

Dockerfiles and container images

Each custom module has a Dockerfile and .dockerignore. The API Dockerfiles use node:20-alpine with base, test and production stages. Dependency manifests are copied before application code to reuse dependency layers.

The API test stages run npm ci and npm test. Production stages install only production dependencies, copy src, expose their internal ports and run as USER node. Each API image checks /health with wget.

The frontend uses node:22-alpine for dependency installation and Vite compilation. Its production stage copies dist into nginxinc/nginx-unprivileged:1.27-alpine. Node and frontend source are omitted from the serving image. The development stage runs Vite separately.

The database Dockerfile extends postgres:16.6-alpine and copies init.sql into docker-entrypoint-initdb.d. Schema and seed initialization run only when the data volume is new. Rebuilding the image does not migrate an existing database.

The Jenkins Dockerfile adds the Docker CLI, Buildx and Compose plugin to the Jenkins LTS JDK17 image. Application configuration is baked into images to avoid Jenkins workspace bind mount path problems.

Image	Runtime detail
catalog-api	3001, /health, USER node
order-api	3002, /health, USER node
payment-api	3003, /health, USER node
frontend	8080, static Nginx health check
proxy	8080, /nginx-health
db	5432, pg_isready in Compose

Docker Compose and startup

docker-compose.yml starts six application infrastructure services: proxy, frontend, three APIs and db. The production proxy is the only service publishing an application port. Database/API ports stay private. Restart policies are set to unless-stopped.

The catalog and order APIs wait for database health. The order API also waits for payment health. The proxy waits for healthy frontend and APIs. PostgreSQL health uses pg_isready; other health checks are declared in the corresponding Dockerfiles.

DB_USER and DB_PASSWORD must be nonempty. .env is ignored by Git and .env.example contains placeholders. scripts/init-env.cjs creates a random password only when no nonempty password exists. Keep the same credentials after the database initializes.

Start production only after stopping host Vite. Jenkins starts from its own Compose file and project. The one application startup command deliberately does not redeploy Jenkins.

```
npm.cmd run repair:env
docker compose up -d --build --wait --wait-timeout 240
npm.cmd run jenkins
npm.cmd run verify

# Local Vite development instead
npm.cmd run install:frontend
npm.cmd run setup:dev
npm.cmd run dev
```

Jenkins installation and configuration

Jenkins runs in infra/jenkins/docker-compose.yml with 8081:8080, a persistent jenkins_home volume and the host Docker socket mounted at /var/run/docker.sock. The agent communication port 50000 is also published by the supplied lab configuration.

Open localhost:8081, complete the unlock/setup flow and create an administrator. Install the suggested plugins, GitHub and Docker Pipeline. Verify Docker access from inside Jenkins using docker exec jenkins docker version.

Add a Secret file credential with ID quickcart-env containing the current PostgreSQL .env. Add repository credentials if the shared repository is private. Never place secret contents in Git, the report or screenshots.

Create a Pipeline script from SCM job using the real shared repository, the agreed deployment branch and Script Path Jenkinsfile. Create another SCM Pipeline with Jenkinsfile.rollback for rollback. An initial setup build registers the pipeline triggers.

githubPush enables GitHub webhook triggering; Poll SCM H/2 * * * * is the fallback. The webhook needs a reachable Jenkins URL, while polling needs Jenkins access to the repository. A successful manual build alone does not meet the automatic deployment requirement.

This classroom Jenkins container runs as root with Docker socket access. That grants broad host Docker control. Explain this lab choice and alternatives such as dedicated build agents during Q&A. Runtime setup and screenshots remain pending.

Pipeline stages and failure behavior

Jenkins sets COMPOSE_PROJECT_NAME to quickcart and TAG to the build number. Compose references TAG for all six images, so a successful build can be redeployed without rebuilding.

The Test stage runs the test target of every API Dockerfile and frontend lint. A failure stops execution before image production builds and deployment. The supplied API route tests mock repositories and therefore do not prove real PostgreSQL connectivity.

Build Images and Deploy obtain .env from Jenkins Credentials. The pipeline removes Windows carriage returns before using Compose. The post section reports success/failure and removes .env from the workspace. Concurrent builds of the main pipeline job are disabled.

A failure after Deploy can leave a partially changed or unhealthy stack. The pipeline reports failure and provides rollback guidance; it does not claim automatic recovery. A separate rollback job must be run with a compatible successful image tag.

Stage	Operation	Failure effect
Checkout	checkout scm	Stops pipeline
Test	API tests and frontend lint	No deployment
Build Images	docker compose build	Previous deployment untouched
Deploy	compose up without build	May partially change runtime
Smoke Test	Frontend, proxy, API and catalog checks	Marks deployed build failed
Post	Report and credential cleanup	Records result

Testing strategy and verified results

On 9 October 2026, the existing seven Jest suites were run locally with --runInBand. All 76 tests passed: catalog 20, order 39, payment 17. Frontend ESLint passed from its frontend directory, and Vite compiled the production bundle successfully.

Catalog tests cover filtering, discounts, suggestions and API responses. Order tests cover totals, validation, checkout success, declined/unavailable payment, unavailable stock, order history and support. Payment tests cover Luhn checks, expiry, CVV and simulated methods.

Route tests replace database/payment repositories with mocks. These are useful quality gates but do not replace a real database integration run. The runtime verifier checks public routes through the proxy, executes PostgreSQL queries and verifies Jenkins can access Docker.

The updated smoke test checks /nginx-health, frontend HTML, all three API health endpoints and a real catalog query. Docker runs and Jenkins execution remain unverified in this assistant environment due to named-pipe access denial.

For the required failed-test demonstration, deliberately change the $10 discount constant to $5 on an agreed demo branch and push. Show the Test stage fail, Deploy skipped and the previous browser build still running. Revert only that intentional demo commit afterward.

Check	Result	Scope
Catalog Jest	20 passed	Local mocked routes and logic
Order Jest	39 passed	Local mocked routes and logic
Payment Jest	17 passed	Local gateway and routes
Frontend lint	Passed	Static code quality
Vite build	Passed	Compiled production assets
Docker/Jenkins	Pending	Host runtime evidence required

Rollback and persistence demonstration

PostgreSQL stores data in the named pg-data volume. docker compose down removes containers and networks while retaining the volume unless -v is used. No data reset is required for normal redeployment. Old MySQL records were not imported automatically.

Create a new order with a unique demo email, record its order number and show it in My Account. Redeploy through Jenkins and find the same order again. Capture before/after screenshots containing the order number and visible build labels. Seeded sample orders alone are insufficient proof of a new successful checkout.

Rollback selects a previous successful PostgreSQL build number. The updated rollback job accepts numeric tags and checks proxy, frontend, all APIs and db images before redeploying. A passing smoke check and restored footer build number demonstrate the result.

Image rollback does not roll back database schema or data. Use only images compatible with the existing PostgreSQL schema, and never choose the older MySQL images. The rollback job and deployment job are separate jobs and must not be run simultaneously.

Actual before/after screenshots and pipeline Stage View remain to be captured on the accessible Docker host. Use docs/demo-runbook.md and preserve pg-data throughout the demonstration.

```
# PowerShell example after a compatible successful build exists
$env:TAG="12"
docker compose -p quickcart up -d --no-build
Remove-Item Env:TAG

# Preserve data when restarting
docker compose down
docker compose up -d --no-build
```

Problems encountered and resolutions

The current code uses PostgreSQL, but supplied Word documentation still described MySQL, root credentials, port 3306 and db-data. This report replaces those details with PostgreSQL 16.6, DB_USER/DB_PASSWORD, port 5432 and pg-data. The original instructor guidelines are retained unchanged.

The historical guide described npm run dev as a full Docker startup. The actual script launches Vite only. GABAY.md now separates development setup, host Vite and production Docker commands, reducing the chance of missing backend services.

Host Vite and the production proxy both use 8080. The host development override moves the Docker proxy to loopback 8082. Stop host Vite before production/Jenkins deployment, which returns the proxy to 8080.

Docker commands in this assistant environment return permission denied for the Docker Desktop Linux named pipe, while the supplied user PowerShell screenshot shows a working Docker Server. Live container evidence must therefore be collected from the accessible host session.

Rollback originally checked only the frontend image. It now validates numeric tags and checks all six application images before deployment. Smoke checks now also cover the frontend and proxy. These changes still require Docker/Jenkins execution testing.

No Git contribution history, deployed Jenkins stages, persistence screenshots or rollback screenshots have been fabricated. The group must record these from real actions before final submission.

Individual responsibilities and references

The roles below are taken from the supplied QuickCart draft. They describe assigned responsibilities rather than verified authorship. Attach each member's real commit or pull request links after the shared repository is confirmed.

Every member must understand the full system for individual Q&A. Discuss which tasks each member actually completed, including fixes, tests, documentation and rehearsals. The instructor needs repository access, and all five members need meaningful contributions.

Final submission requires the technical PDF of 8 to 15 pages, presentation slides of 12 to 20 pages and genuine live demonstration evidence. Rehearse the 10 minute presentation twice, have all five members speak and record a backup video.

Source materials: SYSIN Final Project Guidelines.docx; QuickCart.docx; the supplied QuickCart PostgreSQL project archive; current Dockerfiles, Compose files, Jenkinsfiles, SQL schema and test results.

Reference sites for the group: docs.docker.com; jenkins.io/doc; postgresql.org/docs/16; nginx.org/en/docs; expressjs.com; react.dev; vite.dev; jestjs.io. These are reference links, not evidence that a live run has passed.

Member	Assigned responsibility
Cinense, Kyle Mathew T.	DevOps and Project Manager
Cuenco, John Gabrielle D.	Frontend Lead
Dayrit, Shaira Benneth M.	QA Documentation and Presentation
Frayre, Angeline B.	Database and Orchestration Lead
Perez, Jan Andre T.	Backend Microservice Lead